A Predictive Risk Monitoring and Contribution Analytics for Undergraduate Software Projects

***Abstract*—Undergraduate software projects are commonly supervised through progress meetings, reports, and demonstrations. These methods provide useful context, but they may not reveal prolonged inactivity, delayed tasks, unresolved defects, or uneven participation early enough for intervention. This research presents a Predictive Risk Monitoring and Contribution Analytics component for an Intelligent Project Management System (IPMS). The component combines GitHub development activity with Jira project-management information and provides two complementary views: project-level risk and member-level contribution. Individual participation is represented through a Contribution Index based on Commit Activity, Task Completion, Collaboration, Issue Resolution, and Code Quality. Project risk is classified as High, Medium, or Low using twelve activity, schedule, quality, progress, and team related features. Evaluation was carried out in two stages. A controlled set of 4,000 project-state records was used to compare four classification algorithms, while approximately ten real GitHub repositories were used to verify that the implemented pipeline could retrieve and process genuine repository activity. An integrated GitHub-Jira case was also used to examine task progress, overdue work, delays, and inactivity. Logistic Regression produced the highest accuracy. The real-repository tests were used as operational validation rather than as a claim of real-world classification accuracy. The results show that combining repository activity, task information, and interpretable warning conditions can provide supervisors with a clearer view of project health and observable team participation.**

Index Terms—undergraduate software projects, project risk, contribution analytics, GitHub, Jira, machine learning, Contribution Index, project monitoring.

I. INTRODUCTION

Team-based software projects are an important part of computing education because they require students to apply technical knowledge while planning and coordinating work with others. A typical project involves requirements analysis, implementation, testing, defect handling, documentation, and project management. However, for supervisors, it is difficult to follow several teams at the same time. A project may appear satisfactory during a scheduled meeting while development activity has already slowed, tasks are becoming overdue, or the workload has become concentrated on one member.

Most academic supervision still relies on periodic meetings, progress presentations, reports, and student explanations. These methods remain necessary because they provide context, but they are not continuous. Development tools create a second source of evidence. GitHub records commits, issues, pull requests, reviews, and contributor activity, while Jira records tasks, status changes, priorities, due dates, and completion information. The GitHub REST API supports programmatic retrieval of repository data, and Jira Cloud provides REST resources for projects and issues [8], [9]. This makes it possible to transform day-to-day project activity into monitoring indicators.

Previous studies show that GitHub data can help instructors understand student collaboration and individual participation, but they also show why repository activity should not be treated as a complete measure of effort. Tushev et al. found that student teams use GitHub in different ways and warned that GitHub metrics can be risky when used directly as a proxy for performance [1]. Hundhausen et al. combined GitHub, chat, peer-evaluation, and instructor data when studying individual contribution [2]. More recently, Buffardi et al. examined commits, pull requests, issues, reviews, and comments across 94 undergraduate software engineering teams, showing that contribution patterns also have a consistency dimension [4]. These findings support a multi-dimensional approach rather than a simple commit count. Project risk creates a related but different problem. A project can become difficult to complete because of prolonged inactivity, overdue work, defects, low progress, or repeated delays. Machine-learning methods have been investigated for the assessment and prediction of risk in software projects [5], [6]. However, a useful academic monitoring tool should not only produce a risk label; it should also show the evidence behind that label so that a supervisor can decide whether intervention is necessary.

This research addresses both needs through the Predictive Risk Monitoring and Contribution Analytics component of an Intelligent Project Management System. The component integrates the information from GitHub and Jira, calculates contribution indicators for individual members, classifies project risk, and presents interpretable warnings. The system is intended as decision support rather than an automatic grading mechanism.

The study is guided by three research questions. RQ1 asks how observable GitHub and Jira activity can be combined into a multi-dimensional indicator of individual contribution. RQ2 asks whether project activity, schedule, quality, progress, and team indicators can be used to classify project

risk into High, Medium, and Low categories. RQ3 asks how predicted risk and explicit warning conditions can be presented together so that a supervisor can understand why a project requires attention.

The main contributions are a five-dimensional contribution index, an AHP-based analysis of contribution criteria, a 12feature risk-monitoring model, GitHub-Jira integration, and a dashboard that combines model output with rule-based warning evidence. The evaluation deliberately separates classifier testing from real-repository testing so that model accuracy is not confused with operational validation.

II. RELATED WORK AND RESEARCH GAP

A. Repository Data in Software Engineering Education

Repository histories provide observable evidence of software-development activity. In a case study of a senior software engineering class, Tushev et al. examined GitHub as a collaboration and evaluation tool and reported that teams adapted it differently to their workflows [1]. Their study is important to this research because it shows both the value and the limits of repository metrics in education. Hundhausen et al. studied undergraduate software teams using GitHub activity together with online communication, peer evaluation, and instructor assessment [2]. Their work demonstrates that individual contribution is better understood when several forms of evidence are considered together. Buffardi et al. later analyzed the consistency and parity of contributions for 465 students in 94 teams using commits, pull requests, issues, code reviews, and comments [4]. These studies motivate the use of multiple activity dimensions in the present Contribution Index.

B. Multi-Dimensional Developer Measurement

The SPACE framework argues that developer productivity cannot be represented by a single metric or dimension [3]. Although SPACE was developed for professional software development rather than student grading, the principle is directly relevant: activity measures need context. In an undergraduate project, a member may contribute through code, task completion, issue handling, collaboration, testing, or design. The proposed CI therefore uses several observable dimensions and is presented as supporting evidence rather than a complete measure of academic effort.

C. Software Project Risk Prediction

Machine learning has been used to support softwareproject risk assessment. Panneerselvam and Sankaranarayanan applied machine-learning classifiers to softwareproject risk indicators [5], while Alatawi et al. investigated a data-driven neural-network approach to software-project risk assessment [6]. These studies show that risk can be modeled from multiple project indicators. The present research applies the same general idea to the context of undergraduate software projects, where repository activity and task-management data are available during development.

D. Research Gap

The reviewed work addresses contribution analysis, developer measurement, or software-project risk, but undergraduate supervision often requires these views at the same time. A supervisor may need to know whether a project is becoming risky and whether observable work is distributed across the team. These questions should not be collapsed into one score. The proposed component therefore keeps project-level risk and member-level contribution separate while presenting them in one monitoring workflow.

TABLE I
COMPARISON WITH RELATED WORK

Study	Data / Focus	Contribution	Risk
Tushev et al.[1]	GitHub in SE classes	Partial	No
Hundhausen et al. [2]	GitHub, chat, and peer data	Yes	No
Forsgren et al. [3]	Multi-dimensional productivity	Conceptual	No
Buffardi etal. [4]	GitHub team activity	Yes	No
Panneerselvam et al. [5]	ML-based project risk prediction	No	Yes
Alatawi et al. [6]	Data-driven project risk assessment	No	Yes
Proposed Research	GitHub + Jira monitoring	Yes	Yes

III. RESEARCH METHODOLOGY

A. System Architecture

The component follows a simple processing pipeline: data acquisition, preprocessing, feature extraction, analytics, and presentation. GitHub and Jira provide the source data. The preprocessing layer converts API responses into project and member indicators. The analytics layer calculates the Contribution Index and risk classification. The presentation layer displays risk, progress, activity, contribution, and warning information.

The processing flow is expressed as

- $D \rightarrow P \rightarrow F \rightarrow \{CI, R\} \rightarrow V$

In (1), D represents collected data, P preprocessing, F feature extraction, CI the Contribution Index, R risk classification, and V visualization. The prototype uses a FastAPIbased backend and a short in-memory cache to reduce repeated API calls during dashboard use.

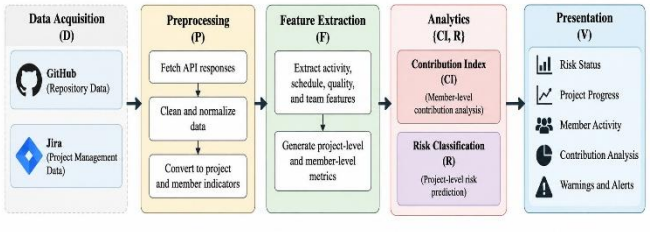


Fig. 1. Architecture of the Predictive Risk Monitoring and Contribution Analytics component.

B. GitHub and Jira Data Integration

GitHub is used to obtain repository and contributor activity such as commits, recent commit activity, issues, pull requests, and collaboration-related events. Jira is used to obtain task and project-management information such as total tasks, completed tasks, overdue tasks, priorities, progress, and delays. The two sources are complementary: GitHub reflects development activity, while Jira provides schedule and task context. The system can still operate with GitHub information when Jira is unavailable, but Jira-dependent

TABLE II
PROJECT-RISK FEATURES

No.	Feature	Purpose
1	Total commits	Overall development activity
2	Weekly commits	Recent development activity
3	Inactive days	Current engagement
4	Overdue tasks	Schedule pressure
5	Bug count	Defect workload
6	High-priority bugs	Severity-related risk
7	Task completion rate	Completed planned work
8	Code quality score	Quality-related proxy
9	Collaboration score	Observable teamwork
10	Contribution Index	Contribution context
11	Project progress	Overall progress
12	Delay count	Repeated schedule delay

indicators are then incomplete. For this reason, GitHub-only output is treated as a reduced monitoring view rather than as equivalent to the full integrated analysis.

C. Project-Risk Features

Twelve features are used to represent the current state of a project. They were selected to cover development activity, schedule, quality, progress, and team behavior.

These features were selected because they represent different risk dimensions. Total and weekly commits represent development activity. Inactive days represent current engagement. Overdue tasks and delay count represent schedule conditions. Bug-related measures provide a quality-oriented view. Task completion and project progress represent delivery status. Collaboration and contribution measures provide team-related context. Continuous features are processed using the stored preprocessing configuration before being passed to the trained model.

D. Contribution Index

The Contribution Index summarizes observable member participation through Commit Activity, Task Completion, Collaboration, Issue Resolution, and Code Quality. Each component is normalized before weighting. The operational formula used by the prototype is:

$$CI_i = 0.30C_i + 0.25T_i + 0.20L_i + 0.15I_i + 0.10Q_i \quad (1)$$

where C_i represents Commit Activity, T_i represents Task Completion, L_i represents Collaboration, I_i represents Issue Resolution, and Q_i represents Code Quality for member i. Commit Activity has the largest weight, but it contributes only 30% of the total score. This prevents the index from becoming a simple commit-count measure.

The contribution share of each member is calculated as:

$$CS_i = \frac{CI_i}{\sum_{j=1}^{n} CI_j} \times 100 \quad (2)$$

TABLE III
CONTRIBUTION INDEX WEIGHTS

Dimension	Operational Weight
Commit Activity	30%
Task Completion	25%
Collaboration	20%
Issue Resolution	15%
Code Quality	10%
Total	100%

where CS_i represents the percentage share of the team's measured contribution for member i. The prototype uses a 60% contribution share as a warning condition for possible imbalance. This warning does not automatically indicate poor performance or free-riding; it indicates that the contribution distribution should be reviewed by a supervisor.

E. AHP-Based Criterion Analysis

The Analytic Hierarchy Process (AHP) provides a structured method for comparing criteria through pairwise

judgments [7]. It was used as a separate analysis of contribution dimensions. The resulting priorities were Commit Activity 22.7Task Completion 21.8Code Quality 19.9Activity Consistency 19.0and Collaboration 16.6These AHP values are reported separately from the operational CI because the criterion sets are not identical: the operational CI contains Issue Resolution, whereas the AHP analysis contains Activity Consistency. The AHP results therefore provide a weighting reference rather than a claim that the deployed CI has already been fully validated.

TABLE IV
AHP-DERIVED CONTRIBUTION PRIORITIES

AHP Criterion	Weight
Commit Activity	22.7%
Task Completion	21.8%
Code Quality	19.9%
Activity Consistency	19.0%
Collaboration	16.6%
Total	100.0%

F. Risk Classification and Warning Rules

Risk classification is treated as a three-class problem in which the model maps the twelve project features to High, Medium, or Low risk. Four algorithms were compared: Logistic Regression, Gradient Boosting, Random Forest, and Decision Tree. f(x) -¿ High, Medium, Low (4) The machinelearning result is combined with explicit monitoring rules. Warnings include high or medium predicted risk, inactivity beyond the configured threshold, no weekly commits, overdue tasks, delayed tasks, high-priority defects, and a contribution share above 60This design keeps the output interpretable because a supervisor can see the project conditions that accompany the prediction.

IV. LIMITATIONS AND THREATS TO VALIDITY

The first limitation concerns the classifier-development data. The 4,000 project states were generated under controlled conditions rather than collected longitudinally from 4,000 completed student projects. The model results are therefore useful for comparing algorithms and testing the analytical pipeline, but they should not be generalized directly to future student cohorts. The second limitation is the absence of independent risk labels for the approximately ten real GitHub repositories. These repositories strengthened the operational evaluation because they provided genuine development activity, but they could not be used to calculate classification accuracy. A stronger future study should collect real student projects together with supervisor-assigned or expert-agreed risk labels. A third limitation concerns construct validity. GitHub and Jira capture only part of student work. Activities such as research, design discussions, documentation, presentations, and offline testing may not be represented. Different teams also use commits, branches, issues, and tasks differently, which can affect measured values. The current CI weights also require further validation. The AHP analysis provides a structured reference, but the operational weights should be compared with independent expert or supervisor judgments before stronger claims are made about their suitability for academic assessment. Finally, the prototype depends on external APIs and their available data. Authentication, rate limits, missing task fields, repository permissions, and inconsistent project configuration can affect the completeness of monitoring information.

V. CONCLUSION AND FUTURE WORK

This paper presented a Predictive Risk Monitoring and Contribution Analytics component for undergraduate software projects. The research addresses two related supervision problems: identifying project-level risk early and understanding the distribution of observable contribution among team members.

The component integrates GitHub and Jira data and transforms them into measurable project and member indicators. Individual contribution is represented through a five-dimensional Contribution Index using Commit Activity, Task Completion, Collaboration, Issue Resolution, and Code Quality. A separate AHP analysis examines the relative importance of contribution criteria and demonstrates that weighting decisions must be treated carefully.

For risk monitoring, twelve project-level features were used to classify projects into High, Medium, and Low risk. Four machine-learning algorithms were evaluated on a

synthetic dataset containing 4,000 records. Logistic Regression achieved the highest recorded test accuracy at 97.63%, followed by Gradient Boosting at 94.00%, Random Forest at 92.38%, and Decision Tree at 90.25%. The selected Logistic Regression experiment also produced a weighted F1-score of approximately 0.98.

The results establish the technical feasibility of the proposed analytical pipeline but do not justify treating the reported accuracy as expected performance on real student projects. The synthetic nature of the dataset remains the most important limitation.

The project demonstration further showed that risk prediction becomes more useful when presented with interpretable evidence. Indicators such as inactivity, delayed tasks, overdue work, recent commits, progress, and contribution imbalance can help a supervisor understand why attention may be required.

Future work should collect real undergraduate project data across several academic periods and evaluate the classifier using class-level precision, recall, F1-score, confusion matrices, and cross-validation. The Contribution

Index should be validated against independent supervisor and expert assessments. Further research should also examine adaptive contribution weights, richer software-quality measures, longitudinal risk trends, missing-data handling, explainable model output, and the inclusion of contribution evidence that occurs outside GitHub and Jira.

The intended role of the system is therefore decision support rather than automated academic judgement. Software activity can provide useful evidence, but human supervision remains necessary to understand the context behind that evidence.

REFERENCES

[1] M. Tushev, G. Williams, and A. Mahmoud, "Using GitHub in large software engineering classes: An exploratory case study," *Computer Science Education*, vol. 30, no. 2, pp. 155–186, 2020, doi: 10.1080/08993408.2019.1696168.

[2] C. Hundhausen, P. Conrad, O. Adesope, and A. Tariq, "Combining GitHub, chat, and peer evaluation data to assess individual contributions to team software development projects," *ACM Transactions on Computing Education*, vol. 23, no. 3, Art. no. 33, 2023, doi: 10.1145/3593592.

[3] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, "The SPACE of developer productivity: There's more to it than you think," *ACM Queue*, vol. 19, no. 1, pp. 20–48, 2021, doi: 10.1145/3454122.3454124.

[4] K. Buffardi, R. Bijoor, A. K. Srivastava, T. K. Sidhu, S. Mamunooru, and A. More, "Chronicling consistency and parity of contributions to software engineering team projects," in Proc. 30th ACM Conf. Innovation and Technology in Computer Science Education (ITiCSE), 2025, pp. 562-568, doi: 10.1145/3724363.3729039.

[5]] G. Panneerselvam and S. Sankaranarayanan, "Prediction of risk percentage in software projects by training machine learning classifiers," Computers Electrical Engineering, vol. 94, Art. 107362, 2021, doi: 10.1016/j.compeleceng.2021.107362.

[6] M. N. Alatawi et al., "A data-driven artificial neural network approach to software project risk assessment," IET Software, vol. 2023, Art. 4324783, 2023, doi: 10.1049/2023/4324783.

[7] T. L. Saaty, "How to make a decision: The Analytic Hierarchy Process," *European Journal of Operational Research*, vol. 48, no. 1, pp. 9–26, 1990, doi: 10.1016/0377-2217(90)90057-I.

[8] GitHub, "GitHub REST API documentation," *GitHub Docs*. [Online]. Available: GitHub REST API documentation. Accessed: Sep. 27, 2026.

[9] Atlassian, "Jira Cloud platform REST API v3," *Atlassian Developer Documentation*. [Online]. Available: Jira Cloud REST API documentation. Accessed: Sep. 27, 2026.

[10] K. K. Aggarwal and Y. Singh, "Software reliability apportionment using the analytic hierarchy process," *ACM SIGSOFT Software Engineering Notes*, vol. 20, no. 5, pp. 56–61, 1995.

[11] M. Pena Gonz˜ alez, C. G. Barrionuevo De La Rosa, and F. J. Cede´ no˜ Moran, "Focus group and Analytic Hierarchy Process for identifying and weighting software project critical success factors," *International Journal of Innovation and Applied Studies*, vol. 15, no. 4, pp. 743–746, 2016.

[12] M. W. Godfrey and Q. Tu, "Evolution in open source software: A case study," in *Proc. IEEE Int. Conf. Software Maintenance*, 2000, pp. 131–142.